

8. Database and Data Modelling

8.1. File Based System

- Data stored in discrete files, stored on computer, and can be accessed, altered or removed by the user

Disadvantages of File Based System:

- No enforcing control on organization/structure of files
- Data repeated in different files; manually change each
- Sorting must be done manually or must write a program
- Data may be in different format; difficult to find and use
- Impossible for it to be multi-user; chaotic
- Security not sophisticated; users can access everything

8.2. Database Management Systems (DBMS)

- **Database:** collection of non-redundant interrelated data
- **DBMS:** Software programs that allow databases to be defined, constructed and manipulated

Features of a DBMS:

- **Data management:** data stored in relational databases - tables stored in secondary storage
- **Data dictionary** contains:
 - List of all files in database
 - No. of records in each file
 - Names & types of each field
- **Data modeling:** analysis of data objects used in database, identifying relationships among them
- **Logical schema:** overall view of entire database, includes entities, attributes and relationships
- **Data integrity:** entire block copied to user's area when being changed, saved back when done

- **Data security:** handles password allocation and verification, backups database automatically, controls what certain user's view by access rights of individuals or groups of users

Data change clash solutions:

- Open entire database in **exclusive mode** – impractical with several users
- **Lock all records** in the table being modified – one user changing a table, others can only read table
- **Lock record** currently being edited – as someone changes something, others can only read record
- User specifies **no locks** – software warns user of simultaneous change, resolve manually
- Deadlock**: 2 locks at the same time, DBMS must recognize, 1 user must abort task

Tools in a DBMS:

- **Developer interface:** allows creating and manipulating database in SQL rather than graphically
- **Query processor:** handles high-level queries. It parses, validates, optimizes, and compiles or interprets a query which results in the query plan.

8.3. Relational Database Modelling

- **Entity:** object/event which can be distinctly identified
- **Table:** contains a group of related entities in rows and columns called an entity set
- **Tuple:** a row or a record in a relational database
- **Attribute:** a field or column in a relational database
- **Primary key:** attribute or combination of them that uniquely define each tuple in relation
- **Candidate key:** attribute that can potentially be a primary key
- **Foreign key:** attribute or combination of them that relates 2 different tables
- **Referential integrity:** prevents users or applications from entering inconsistent data
- **Secondary key:** candidate keys not chosen as the primary key
- **Indexing:** creating a secondary key on an attribute to provide fast access when searching on that attribute; indexing data must be updated when table data changes

Advantages of Relational Database Modelling

- **Data Independence:** The database structure (schema) is separate from the application programs. Changes to the database layout do not break software functionality.
- **Controlled Redundancy:** Data is stored in one central location and referenced elsewhere using links, preventing unnecessary duplication.

- **Data Integrity:** Centralized updates ensure data consistency. Enforced validation rules (e.g., validation checks, foreign keys) maintain accuracy.
- **Centralized Security:** The Database Management System (DBMS) controls access rights, allowing specific users access to only specific tables or columns.
- **Data Sharing:** Multiple applications and users can access the same centralized data concurrently.
- **Concurrency Control:** The DBMS uses locking mechanisms to prevent data corruption when multiple users update data simultaneously.

8.4. Relational Design of a System



8.5. Normalization

1st Normal Form (1NF): contains no repeating attribute or groups of attributes. Intersection of each tuple and attribute contains only 1 value. Example:

DELNOTE

Num	CustName	City	Country	ProdID	Description
005	Bill Jones	London	England	1	Table
005	Bill Jones	London	England	2	Desk
005	Bill Jones	London	England	3	Chair
008	Mary Hill	Paris	France	2	Desk
008	Mary Hill	Paris	France	7	Cupboard
014	Anne Smith	New York	USA	5	Cabinet
002	Tom Allen	London	England	7	Cupboard
002	Tom Allen	London	England	1	Table
002	Tom Allen	London	England	2	Desk

2nd Normal Form (2NF): it is in 1NF and every non-primary key attribute is fully dependent on the primary; all the incomplete dependencies have been removed. Example:

DELNOTE

Num	CustName	City	Country
005	Bill Jones	London	England
008	Mary Hill	Paris	France
014	Anne Smith	New York	USA
002	Tom Allen	London	England

DEL_PROD

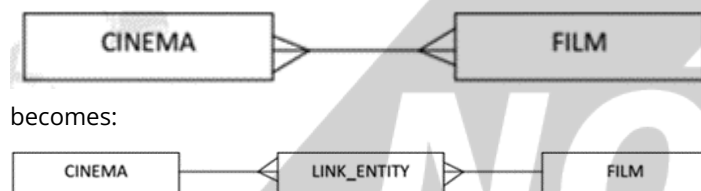
Num	ProdID
005	1
005	2
005	3
008	2
008	7
014	5
002	7
002	1
002	2

PRODUCT

ProdID	Description
1	Table
2	Desk
3	Chair
7	Cupboard
5	Cabinet

3rd Normal Form (3NF): it is in 1NF and 2NF and all non-key elements are fully dependent on the primary key. No inter-dependencies between attributes.

- MANY-TO-MANY functions cannot be directly normalized to 3NF, must use a 2 step process e.g.



8.6. Data Definition Language (DDL)

- Creation/modification of the database structure using this language
 - written in SQL
- Creating a database:

```
CREATE DATABASE <database-name>
```

- Creating a table:

```
CREATE TABLE <table-name> (...)
```

- Changing a table:

```
ALTER TABLE <table-name>
```

- Adding a primary key:

```
PRIMARY KEY (field)
```

```
ADD <field-name>:<data-type>
```

- Adding a foreign key:

```
FOREIGN KEY (field) REFERENCES <table>(field)
```

- Example:

```
CREATE DATABASE 'Personnel.gdb'
CREATE TABLE Training
(EmpID INT NOT NULL,
 CourseTitle VARCHAR(30) NOT NULL,
 CourseDate Date NOT NULL,
 PRIMARY KEY (EmpID, CourseDate),
 FOREIGN KEY (EmpID) REFERENCES Employee(EmpID))
```

8.7. Data Manipulation Language (DML)

- Query and maintenance of data done using this language – written in SQL

Queries

- Creating a query:

```
SELECT <field-name>
FROM <table-name>
WHERE <search-condition>
```

- SQL Operators:

=	Equals to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
<>	Not equal to
IS NULL	Check for null values

- Sort into ascending order:

```
ORDER BY <field-name>
```

- Arrange identical data into groups:

```
GROUP BY <field-name>
```

- Joining together fields of different tables:

```
INNER JOIN
```

Data Maintenance

- Adding data to table:

```
INSERT INTO <table-name>(field1, field2, field3)
VALUES (value1, value2, value3)
```

- Deleting a record:

```
DELETE FROM <table-name>  
WHERE <condition>
```

- **Updating a field in a table:**

```
UPDATE <table-name>  
SET <field-name> = <value>  
WHERE <condition>
```

